

Mise en place d'une suite de tests automatisés

Projet SentimentIA, API d'analyse de sentiments

Rapport d'atelier

Tests unitaires, tests d'integration, integration continue, TDD, tests de performance et audit de securite

1. Introduction et contexte

Dans le cadre de cet atelier, j'ai pris en charge la mise en place de la suite de tests complete de SentimentIA, une API d'analyse de sentiments destinee a traiter les avis clients. Le travail s'est deroule en cinq etapes successives : la mise en place de l'environnement (TP0), l'ecriture des tests unitaires (TP1), les tests d'integration de l'API (TP2), la construction d'un pipeline d'integration continue (TP3), puis un cycle TDD accompagne de tests de performance et d'un audit de securite (TP4).

L'objectif n'etait pas de developper l'application elle-meme, fournie au depart, mais d'en garantir la fiabilite par une strategie de test progressive. J'ai structure ma demarche pour que chaque niveau de test reponde a une question precise : la logique metier est-elle correcte, l'API repond-elle bien, le code reste-t-il sain, et tient-il la charge ? Au fil de l'atelier, quatre defaults ont ete identifies puis corriges grace aux tests.

2. Presentation de l'application

SentimentIA expose une API HTTP construite avec FastAPI. Le coeur metier, la classe `SentimentModel`, analyse un texte en comptant les mots positifs et negatifs qu'il contient, puis en deduit un label (POSITIVE, NEGATIVE ou NEUTRAL) et un score de confiance compris entre 0 et 1. La validation des donnees entrantes est assuree par Pydantic : un texte vide, trop long ou absent est automatiquement rejete. L'API met a disposition quatre points d'entree :

Methode	URL	Role
GET	/health	Verifier que l'API est operationnelle
POST	/predict	Analyser un texte et renvoyer le sentiment
GET	/stats	Consulter les compteurs de predictions
POST	/reset	Remettre les compteurs a zero

3. TP0, mise en place de l'environnement

J'ai cree l'arborescence du projet en separant le code source (src/) des tests (tests/), isole les dependances dans un environnement virtuel, installe les bibliotheques en versions epinglees, et centralise la configuration de pytest dans `pytest.ini`. Les deux smoke tests fournis passent, ce qui confirme que l'environnement est fonctionnel.

Question 0.1. Que renvoie l'API pour un texte vide, un texte de 6000 caracteres, ou un champ text absent ?

Dans les trois cas, l'API renvoie un code 422. Le champ text est contraint par Pydantic avec `min_length=1` et `max_length=5000` : un texte vide viole la longueur minimale, un texte de 6000 caracteres depasse la longueur maximale, et l'absence du champ viole l'obligation de le fournir. Ces rejets ont lieu avant meme que la logique metier soit atteinte.

Question 0.2. Tracez l'exécution de `predict('Ce produit est horrible et mauvais')`.

La tokenisation donne ['ce', 'produit', 'est', 'horrible', 'et', 'mauvais']. Le comptage trouve 0 mot positif et 2 mots négatifs (horrible et mauvais), donc `neg` supérieur à `pos`. Le score vaut $\min(0.6 + 0.1 \times 2, 1.0) = 0.8$. La méthode renvoie `{'label': 'NEGATIVE', 'score': 0.8, 'text': 'Ce produit est horrible et mauvais'}`.

Question 0.3. Dans quel cas `predict()` lève-t-elle une exception ?

Elle lève `SentimentError` lorsque la tokenisation ne trouve aucun mot, c'est à dire quand le texte ne contient aucun caractère alphabétique. Par exemple `predict('1234 !!! ???')` déclenche l'exception. On le reconnaît dans le code au bloc `'if not tokens'` placé juste après la tokenisation.

Question 0.4. Que se passe-t-il si `model.predict()` lève une `SentimentError` dans l'endpoint `/predict` ?

L'endpoint capture l'exception et la transforme en `HTTPException` avec le code 422. C'est donc le même code que celui renvoyé par Pydantic en cas de validation échouée, mais l'origine diffère : ici l'erreur vient de la logique métier, et non de la validation du schéma d'entrée.

Question 0.5. Capture d'écran des 2 smoke tests en statut PASSED.

Les deux smoke tests (`test_health_returns_ok` et `test_predict_positive_text`) passent. `pytest` affiche '2 passed', ce qui valide l'installation de l'environnement.

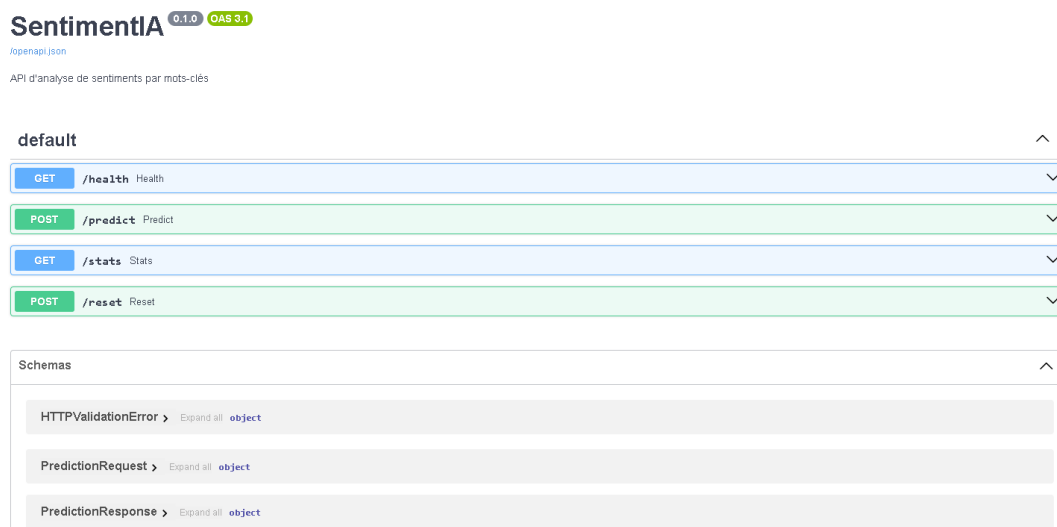


Figure 0.1 – Résultat des 2 smoke tests en vert

Question 0.6. Quelles lignes de `src/model.py` ne sont pas couvertes par les 2 smoke tests, et pourquoi ?

Avant l'écriture des tests du TP1, la couverture laissait de côté la levée de `SentimentError` ainsi que les branches `NEGATIVE` et `NEUTRAL`. C'est normal : les smoke tests n'envoient qu'un texte positif valide, donc seul ce chemin est parcouru. Pour couvrir le reste, il faut des tests unitaires ciblant chaque branche (texte négatif, texte neutre, texte sans mot analysable).

4. TP1, tests unitaires du modèle

Les tests unitaires ciblent la methode `predict()` en isolation. J'ai introduit une fixture qui fournit une instance fraiche du modele a chaque test, couvert les trois labels, l'exception, et utilise la parametrisation ainsi qu'un mock.

Question 1.1. Les deux tests fournis passent-ils ? Que remarquez-vous sur leur nom ?

Les deux tests passent. Leur nom explicite (par exemple `test_predict_retourne_positive`) permet de comprendre immediatement ce qui est verifie et de localiser un echec dans le rapport sans avoir a lire le corps du test.

Question 1.2. Quel est l'apport de la fixture `model()` ?

Les tests ne contiennent plus la ligne de creation de l'instance, elle est injectee automatiquement. Si le constructeur de la classe evolue, je n'ai qu'un seul endroit a modifier au lieu de chaque test. Chaque test démarre aussi avec une instance propre, ce qui evite tout etat partage involontaire.

Question 1.3. Combien de tests genere le bloc parametrize, et comment sont-ils nommes ?

Le bloc avec cinq jeux de donnees genere cinq tests distincts. Chacun apparait individuellement dans le rapport, avec ses parametres entre crochets, par exemple `test_predict_labels_parametrises[produit super-POSITIVE]`. Chaque cas peut donc echouer independamment des autres.

Question 1.4. La vraie methode `predict()` s'execute-t-elle dans le test avec mock ? A quoi sert ce test ?

Non, la vraie methode ne s'execute pas : le mock la remplace et renvoie directement une valeur predefinie. Ce test verifie le contrat d'appel (`predict` appelee une seule fois, avec le bon argument). Ce mecanisme deviendrait indispensable si la methode appelait une API externe payante : on testerait le comportement sans declencher d'appel reseau reel.

Question 1.5. Quelle est votre couverture sur `src/model.py` ?

Apres l'ecriture de tous les tests et la correction du defaut de score, la couverture de `src/model.py` atteint 100 pour cent. Toutes les branches sont exercees : positif, negatif, neutre, exception, et le plafonnement du score.

```
PS C:\Users\bchao\development\automatisation_test\sentiment-ai> pytest tests/test_model.py
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.2.0, pluggy-1.6.0 -- C:\Users\bchao\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\bchao\development\automatisation_test\sentiment-ai
configfile: pytest.ini
plugins: anyio-4.12.1, dash-2.17.1, Faker-24.0.0, cov-5.0.0
collected 27 items

tests/test_model.py::test_predict_retourne_positive PASSED [ 3%]
tests/test_model.py::test_predict_retourne_les_trois_champs PASSED [ 7%]
tests/test_model.py::test_predict_retourne_negative PASSED [ 11%]
tests/test_model.py::test_predict_retourne_neutral PASSED [ 14%]
tests/test_model.py::test_predict_leve_sentiment_error_sur_texte_sans_mots PASSED [ 18%]
tests/test_model.py::test_predict_leve_sentiment_error_sur_texte_vide_de_lettres PASSED [ 22%]
tests/test_model.py::test_predict_labels_parametrises[produit super-POSITIVE] PASSED [ 25%]
tests/test_model.py::test_predict_labels_parametrises[vraiment nul-NEGATIVE] PASSED [ 29%]
tests/test_model.py::test_predict_labels_parametrises[bon et fiable-POSITIVE] PASSED [ 33%]
tests/test_model.py::test_predict_labels_parametrises[lent et mauvais-NEGATIVE] PASSED [ 37%]
tests/test_model.py::test_predict_labels_parametrises[recu hier-NEUTRAL] PASSED [ 40%]
tests/test_model.py::test_predict_score_ne_depasse_pas_1 PASSED [ 44%]
tests/test_model.py::test_predict_score_negatif_borne PASSED [ 48%]
tests/test_model.py::test_predict_equilibre_retourne_neutral PASSED [ 51%]
tests/test_model.py::test_predict_est_appele_avec_le_bon_argument PASSED [ 55%]
tests/test_model.py::test_predict_conserve_le_texte_original PASSED [ 59%]
tests/test_model.py::test_predict_mot_positif_en_majuscules_detecte PASSED [ 62%]
tests/test_model.py::test_predict_score_minimum_un_mot PASSED [ 66%]
tests/test_model.py::test_negation_pas_bien_retourne_negative PASSED [ 70%]
tests/test_model.py::test_negation_jamais_mauvais_retourne_positive PASSED [ 74%]
tests/test_model.py::test_negation_sans_negation_reste_inchange PASSED [ 77%]
tests/test_model.py::test_negation_parametrisee[pas bien-NEGATIVE] PASSED [ 81%]
tests/test_model.py::test_negation_parametrisee[jamais mauvais-POSITIVE] PASSED [ 85%]
tests/test_model.py::test_negation_parametrisee[produit pas nul-POSITIVE] PASSED [ 88%]
tests/test_model.py::test_negation_parametrisee[plus jamais super-NEGATIVE] PASSED [ 92%]
tests/test_model.py::test_negation_parametrisee[sans aucun defaut-NEUTRAL] PASSED [ 96%]
tests/test_model.py::test_negation_parametrisee[vraiment excellent-POSITIVE] PASSED [100%]

===== 27 passed in 0.65s =====
```

Figure 1.1 – Rapport de couverture TP1: 100%

Defaut detecte au TP1 : le score non borne

Un test envoyant les dix mots positifs verifiait que le score ne dépassait pas 1.0. Il a echoue en affichant 1.6 : la formule $0.6 + 0.1 \times n$ n'était pas plafonnee. J'ai corrige le calcul en l'encadrant par `min(..., 1.0)`.

5. TP2, tests d'integration de l'API

Les tests d'integration verifient le comportement de bout en bout, de la requete HTTP a la reponse JSON. J'ai regroupe les outils communs dans `conftest.py` : un client reutilisable, des generateurs de textes via `Faker`, et une fixture de remise a zero des compteurs.

Question 2.1. Pourquoi `reset_stats` utilise `yield`, et pourquoi `autouse=True` ?

`yield` separe ce qui s'execute avant le test de ce qui s'executerait apres. La remise a zero est placee avant le `yield` pour que chaque test démarre d'un etat propre. Si on appelait `/reset` apres le `yield`, la remise a zero aurait lieu a la fin et le premier test partirait d'un etat non garanti. `autouse=True` garantit que cette remise a zero s'applique a tous les tests sans avoir a la declarer, assurant leur isolation systematique.

Question 2.2. Pourquoi `test_stats_incremente` n'appelle pas `/reset`, et que se passe-t-il sans `autouse` ?

Il n'a pas besoin d'appeler `/reset` car la fixture `autouse` l'a deja fait avant le test : les compteurs sont a zero. Sans `autouse`, la fixture ne s'executerait plus automatiquement, les compteurs conserveraient les valeurs des tests precedents, et l'assertion `total_predictions == 0` echouerait de facon imprevisible selon l'ordre d'execution.

Question 2.3. Quelle structure JSON renvoie une erreur 422, et où se trouve le message ?

FastAPI renvoie une structure de la forme `{'detail': [ {...} ]}`. Le message lisible se trouve dans `detail[0]['msg']`, et le champ `detail[0]['loc']` indique précisément quel champ est en cause.

Question 2.4. Pourquoi les tests restent stables malgré Faker, et quel est son intérêt ?

Les fixtures injectent toujours un mot-cle connu du modèle dans une phrase aléatoire, et les assertions portent sur le label (déterminé par ce mot-cle), pas sur le texte exact. Le résultat reste donc déterministe. L'intérêt de Faker est de tester une grande variété de textes réalistes plutôt qu'une seule chaîne figée, ce qui révèle des bugs liés à des formats inattendus.

Question 2.5. Que constatez-vous en inversant l'ordre des deux tests defectueux ?

Selon l'ordre, l'un des deux échoue : le test qui suppose des compteurs à zéro ne réussit que si l'autre ne s'est pas exécuté avant. Comme l'ordre d'exécution de pytest n'est pas garanti, cette dépendance produit des échecs imprévisibles. La fixture autouse de remise à zéro résout le problème en rendant chaque test indépendant.

Question 2.6. À quoi servent les balises de report.xml et pourquoi Jenkins en a besoin ?

Le fichier contient une balise `testsuite` (résumé global) et une balise `testcase` par test, dont l'attribut `time` donne la durée d'exécution. Jenkins a besoin de ce format XML structuré plutôt que de la sortie console car il peut le parser de façon fiable, marquer le build en succès ou en échec, et archiver l'historique sur plusieurs builds pour suivre les tendances (tests qui ralentissent, nouveaux échecs).

```
PS C:\Users\bchao\development\automatisation_test\sentiment-ai> pytest tests/test_api.py
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.2.0, pluggy-1.6.0 -- C:\Users\bchao\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\bchao\development\automatisation_test\sentiment-ai
configfile: pytest.ini
plugins: anyio-4.12.1, dash-2.17.1, Faker-24.0.0, cov-5.0.0
collected 15 items

tests/test_api.py::test_health_returns_ok PASSED [ 6%]
tests/test_api.py::test_predict_positive_text PASSED [ 13%]
tests/test_api.py::test_predict_retourne_structure_complete PASSED [ 20%]
tests/test_api.py::test_stats_incremente_apres_predict PASSED [ 26%]
tests/test_api.py::test_health_retourne_status_ok PASSED [ 33%]
tests/test_api.py::test_predict_avec_texte_positif PASSED [ 40%]
tests/test_api.py::test_predict_avec_texte_negatif PASSED [ 46%]
tests/test_api.py::test_predict_conserve_le_texte_original PASSED [ 53%]
tests/test_api.py::test_stats_contient_les_quatre_champs PASSED [ 60%]
tests/test_api.py::test_stats_comptabilise_plusieurs_predictions PASSED [ 66%]
tests/test_api.py::test_reset_remet_les_compteurs_a_zero PASSED [ 73%]
tests/test_api.py::test_predict_texte_vide_retourne_422 PASSED [ 80%]
tests/test_api.py::test_predict_texte_trop_long_retourne_422 PASSED [ 86%]
tests/test_api.py::test_predict_champ_manquant_retourne_422 PASSED [ 93%]
tests/test_api.py::test_predict_mauvais_type_retourne_422 PASSED [100%]

===== 15 passed in 0.49s =====
```

Figure 2.1 – Rapports JUnit des tests d'intégration

Defaut détecté au TP2 : l'état partagé

Deux tests volontairement dépendants donnaient des résultats différents selon leur ordre d'exécution. La fixture autouse de remise à zéro corrige le problème en isolant chaque test.

6. TP3, pipeline d'intégration continue

J'ai conteneurisé l'application avec un Dockerfile (python:3.12-slim), dont la construction aboutit sans erreur, décrit l'orchestration dans docker-compose, et écrit un Jenkinsfile de six étapes : Checkout, Install, Lint, Tests Unitaires, Tests Integration et Security Scan.

Question 3.1. Pourquoi installer seulement le client Docker dans Jenkins, et quel est le rôle de docker.sock ?

Le client envoie les commandes, le daemon les exécute. On installe seulement le client dans le conteneur Jenkins car il réutilise le daemon de la machine hôte, accessible via le montage du fichier /var/run/docker.sock (le socket par lequel le client dialogue avec le daemon). Cela évite un daemon imbriqué et reste la méthode standard dite Docker-outside-of-Docker.

Question 3.2. Pourquoi la commande Pylint se termine par || true ?

|| true force le code de retour à zéro, donc le stage ne casse jamais sur un simple avertissement de style. Sans ce suffixe, un score inférieur à 7.0 (via --fail-under) renverrait un code non nul, ferait échouer le stage, et par le principe du fail fast stopperait tous les stages suivants.

Question 3.3. Que signifie post { always } et pourquoi publier le rapport JUnit même en cas d'échec ?

always s'exécute quel que soit le résultat du stage, contrairement à success ou failure qui sont conditionnels. On publie le rapport JUnit même quand des tests échouent car c'est précisément dans ce cas qu'on a besoin de voir quels tests ont échoué et pourquoi.

Question 3.4. Pourquoi pip3 --break-system-packages est-il nécessaire et acceptable ici ?

Les distributions récentes protègent l'environnement Python système (PEP 668) et refusent une installation pip globale sans cette option. Dans un conteneur jetable et isolé, il n'y a aucun risque de casser un autre projet, donc installer dans l'environnement système est parfaitement acceptable.

Question 3.5. Pourquoi séparer tests unitaires et tests d'intégration en deux stages ?

Cette séparation permet de savoir immédiatement quel type de test casse (la logique métier ou l'API), de produire deux rapports JUnit distincts, et grâce au fail fast d'éviter de lancer les tests d'intégration, plus lourds, si les tests unitaires échouent déjà.

Question 3.6. Combien d'alertes Bandit obtenez-vous, et de quel niveau ?

Sur mon code source, Bandit ne remonte aucune alerte (0 Low, 0 Medium, 0 High). Le défaut intentionnel évoqué par l'atelier provient d'une variante vulnérable du code (par exemple une écoute réseau sur 0.0.0.0 écrite en Python), que mon implémentation ne contient pas.

Question 3.7. Corrigez les alertes Bandit et confirmez 0 alerte HIGH ou MEDIUM.

Aucune alerte HIGH ou MEDIUM n'est à corriger, l'audit est déjà à zéro. L'écoute sur 0.0.0.0, nécessaire au fonctionnement dans un conteneur, est confinée au Dockerfile, hors du périmètre analyse (src/). En relançant 'bandit -r src/', le résultat reste à 0 alerte.

Question 3.8. Quel score Pylint obtenez-vous une fois le pipeline vert ?

Pylint affiche un score de 9.61 sur 10, nettement au-dessus du seuil de 7.0 exigé par le stage Lint. Le pipeline conserve donc ses six étapes au vert.

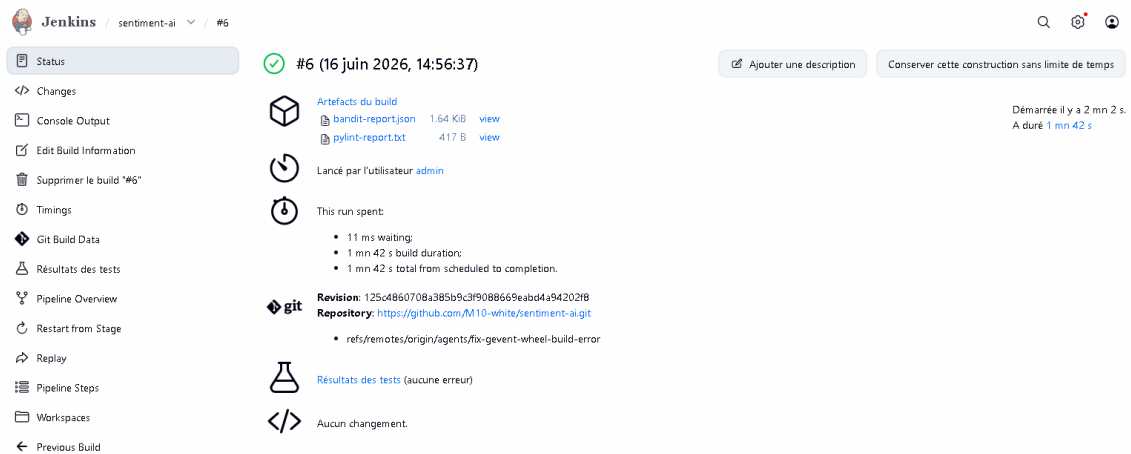


Figure 3.1 – Pipeline Jenkins avec 6 étapes vertes

```
PS C:\Users\bchao\development\automatisation_test\sentiment-ai> pylint src/model.py src/main.py --exit-zero
***** Module src.model
src\model.py:12:0: R0903: Too few public methods (1/2) (too-few-public-methods)
***** Module src.main
src\main.py:1:0: C0114: Missing module docstring (missing-module-docstring)

-----
Your code has been rated at 9.69/10
```

Figure 3.2 – Score Pylint : 9.69/10

7. TP4, TDD, performance et securite

J'ai traite la gestion des negations en suivant le cycle TDD, mene des tests de charge avec Locust, et relance l'audit de securite.

Question 4.1. Pourquoi confirmer que les tests echouent avant d'ecrire le code ?

Confirmer l'echec prouve que le test verifie reellement la nouvelle fonctionnalite : il echoue parce que le comportement n'existe pas encore. Un test qui passerait avant meme d'ecrire le code serait trompeur, signe que ses assertions sont fausses ou qu'il ne teste pas ce que l'on croit.

Question 4.2. Les tests du TP1 passent-ils toujours apres l'ajout des negations ?

Oui, aucune regression. Le fichier test_model.py compte 27 tests au vert (18 du TP1 et 9 lies aux negations). L'implementation des negations n'a casse aucun comportement existant.

Question 4.3. Ajoutez un test parametree couvrant au moins 4 cas de negation.

J'ai ecrit test_negation_parametree avec six cas, qui passe apres le refactor sans modifier le code de production : 'pas bien' donne NEGATIVE, 'jamais mauvais' donne POSITIVE, 'produit pas nul' donne POSITIVE, 'plus jamais super' donne NEGATIVE, 'sans aucun default' donne NEUTRAL, et 'vraiment excellent' reste POSITIVE.

Question 4.4. Votre couverture a-t-elle augmente ou diminue par rapport au TP1 ?

La couverture de src/model.py reste a 100 pour cent, alors que le nombre d'instructions est passe de 24 a 37. Les nouvelles lignes (parcours des tokens, gestion du drapeau de negation, branches inversees) sont toutes exercees par les tests de negation, donc aucune nouvelle ligne non couverte.

Question 4.5. Pourquoi un poids de 3 pour /predict et de 1 pour /stats dans Locust ?

Le poids represente la frequence relative de selection d'une tache par un utilisateur virtuel. Un poids de 3 contre 1 signifie que /predict est appele environ trois fois plus souvent que /stats. Ce ratio reflete un usage reel, ou l'on analyse des textes bien plus frequemment qu'on ne consulte les statistiques.

Question 4.6. Resultats a 50 utilisateurs : le P95 de /predict est-il inferieur a 500 ms ?

Oui, tres largement. Voici les mesures relevees a 50 utilisateurs :

Endpoint	Requetes	P50	P95	Echecs
POST /predict	2162	4 ms	7 ms	0
GET /stats	498	3 ms	6 ms	0

Question 4.7. Comment evolue la latence entre 50 et 100 utilisateurs ?

En doublant la charge, le P95 de /predict passe de 7 ms a 14 ms, sans aucune erreur supplementaire. L'API evolue de facon lineaire sur cette plage et dispose d'une marge confortable avant saturation.

Charge	Requetes	P95 /predict	Echecs
50 utilisateurs	2 660	7 ms	0
100 utilisateurs	4 993	14 ms	0

Question 4.8. Combien d'alertes Bandit subsistent apres les modifications TDD ?

Aucune. L'ajout de la logique de negation n'a introduit ni eval, ni entree non validee, ni secret. Bandit reste a zero alerte et le code source demeure sain.

Question 4.9. Le pipeline reste-t-il vert, et pourquoi le TDD ameliore la couverture ?

Les six etapes restent vertes apres le commit. Le TDD ameliore mecaniquement la couverture car chaque test ecrit avant le code force l'execution de lignes nouvelles, qui passent de non couvertes a couvertes des qu'elles sont implementees.

```
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-8.2.0, pluggy-1.6.0 -- C:\Users\bchao\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\bchao\development\automatisation_test\sentiment-ai
configfile: pytest.ini
plugins: anyio-4.12.1, dash-2.17.1, Faker-24.0.0, cov-5.0.0
collected 6 items

tests/test_model.py::test_negation_parametrisee[pas bien-NEGATIVE] PASSED [ 16%]
tests/test_model.py::test_negation_parametrisee[jamais mauvais-POSITIVE] PASSED [ 33%]
tests/test_model.py::test_negation_parametrisee[produit pas nul-POSITIVE] PASSED [ 50%]
tests/test_model.py::test_negation_parametrisee[plus jamais super-NEGATIVE] PASSED [ 66%]
tests/test_model.py::test_negation_parametrisee[sans aucun default-NEUTRAL] PASSED [ 83%]
tests/test_model.py::test_negation_parametrisee[vraiment excellent-POSITIVE] PASSED [100%]

===== 6 passed in 0.28s =====
```

Figure 4.1 – Tests de negation au vert

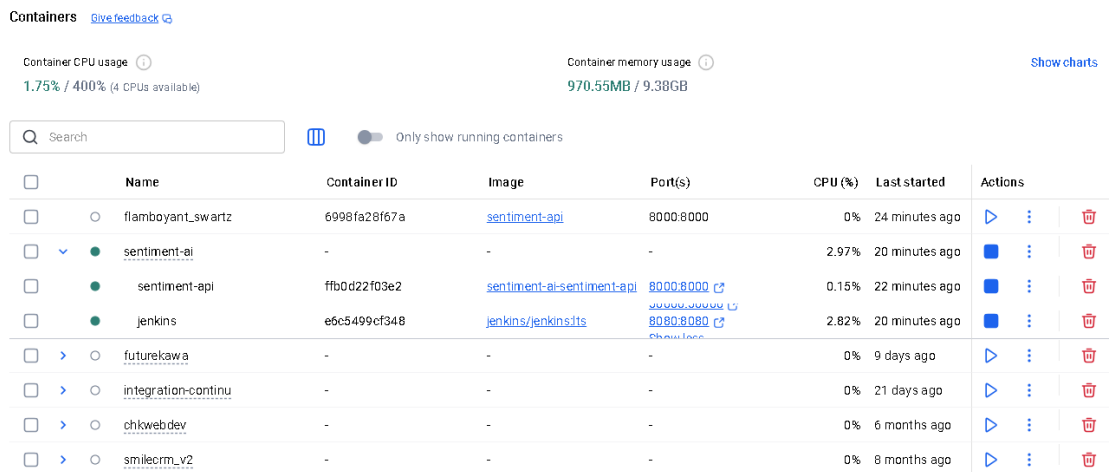


Figure 4.3 – Pipeline final vert après TP4

Question 4.10. Recapitulatif des quatre défauts identifiés et corrigés.

Le tableau ci-dessous resume les quatre défauts rencontrés au fil de l'atelier et la correction que j'ai appliquée à chacun.

Etape	Defaut	Comment je l'ai detecte	Correction apportee
TP1	Score depassant 1.0	Test verifiant score <= 1.0	Plafonnement par min(..., 1.0)
TP2	Tests dependants entre eux	inversion de l'ordre d'execution	Fixture de reset automatique (autouse)
TP3	Qualite et securite du code	PyLint et Bandit dans le pipeline	Code maintenu propre (9.61/10, 0 alerte)
TP4	Negations ignorees	Test TDD en phase RED	Parcours des tokens avec drapeau de negation

8. Conclusion

Au terme de l'atelier, le projet dispose d'une suite de 42 tests, tous au vert, et d'une couverture globale de 98 pour cent (100 pour cent sur la logique métier). L'application est conteneurisée, passe les audits de qualité et de sécurité, et tient la charge avec une latence très faible. L'ensemble s'intègre dans un pipeline automatisé reproductible.

J'ai surtout retenu que les tests ne servent pas seulement à confirmer ce qui fonctionne, mais qu'ils revelent activement les défauts. Cette progression, du test unitaire le plus fin jusqu'au pipeline complet, m'a permis de comprendre concrètement comment une équipe garantit la fiabilité d'un logiciel avant de le livrer, et pourquoi chaque test doit rester indépendant, lisible et reproductible.